MASTER REFERENCE

REMANON

A GPU-resident shared-memory runtime for multi-agent, multi-model AI inference.

"Maximum alignment, minimum detachment."

ONE MEMORY · MANY MINDS · ZERO EVICTIONS

The Complete Reference

The problem, the solution, the originality, the architecture, the measured evidence, and the answers to the hardest questions a technical panel can ask. Every figure states its provenance. Nothing is fabricated.

AMD Developer Hackathon: ACT II · Track 3 — Unicorn
Solo participant: **Nevine Fakhreddin**
github.com/NevineAKF/remanon · Live: nevineakf.github.io/remanon · License: MIT

00 How to read this document

This is the single, complete reference for Remanon. It captures what the system is, why it matters, how it works, what was proven on real hardware, and what was projected by transparent arithmetic from those measurements. It is written to be read by three audiences at once: the engineer who wants to see the mechanism, the investor who wants to see the market, and the researcher who wants to see the method.

Every quantitative claim in this document carries a provenance label. MEASURED means the number was captured on live AMD hardware. COMPUTED means it was projected, by stated and transparent arithmetic, from measured numbers to the target hardware. Nothing is asserted as measured that was not measured. That discipline is the spine of the project, and it is what makes every figure here defensible under scrutiny.

01 What Remanon is

When several AI agents investigate the same problem, they share most of the same information: the same incident, the same logs, the same body of evidence. Yet in every inference stack deployed today, each agent rebuilds that shared context from scratch inside GPU memory, paying again for work another agent has already done moments earlier.

Remanon ends that waste. It materializes the shared context exactly once per distinct model, pins it in high-bandwidth memory so it can never be evicted under any pressure, and lets every agent of that model read the same resident copy while paying only for its own small private delta. This is not a caching optimization dressed up in new language. It is a cross-engine memory arbiter with a contract-enforced residency guarantee, filling a role that no component in the current inference stack occupies.

Attribute	Value
Category	AI infrastructure. A cross-engine shared-context runtime and memory arbiter that sits between agent frameworks and the GPU.
Tagline	Maximum alignment, minimum detachment.
Headline	One memory. Many minds. Zero evictions.
Domain today	AIOps, incident root-cause analysis. The core itself is domain-agnostic.
Hardware target	AMD Instinct MI300X, 192 GB HBM3. Measured on AMD gfx1100, 48 GB, Radeon PRO W7900 class.
Delivery	MIT license, public repository, solo build, live deployed site.

02 The problem — the memory wall of multi-agent AI

A single production incident produces thousands of log lines. A team of specialized agents must all read that same context to investigate it. Without a shared layer, the waste compounds in three directions at once.

There is a waste of **time**, because every agent recomputes a prefill of context that was already computed by another. There is a waste of **memory**, because the GPU fills with duplicate copies of an identical context. And most decisively, there is a **capacity wall**, because that very duplication is what makes a multi-model team overflow the card and fail to load at all. On a fixed memory budget, duplication is not an inefficiency to be tuned away later. It is the wall that decides whether a multi-agent team can run in the first place.

03 The solution — with and without Remanon

	Without Remanon	With Remanon
Shared context	Each agent builds its own private copy	Materialized once per model, shared by all
Same-model agents	Rebuild the same cache separately	Read one pinned master
Under memory pressure	Blocks are silently evicted	The pinned master is never evicted
Multi-model team	Overflows the card and fails	Fits, because duplication is removed
Prefill cost	One prefill per agent	One prefill per distinct model

> **Measured on AMD.** Shared-context reuse runs **5.8 times faster** from cold to warm, a result captured live on the hardware and confirmed independently by the engine's own prefix-cache hit-rate counter. The mechanism is not asserted. It is observed. MEASURED

04 Originality — why this is not vLLM prefix caching

This is the originality-screening surface, the first question any informed panel will raise. The difference is one of contract, not of mechanism.

	vLLM automatic prefix cache	Remanon
Nature	Opportunistic, best-effort	Contract-enforced guarantee
Eviction	May silently evict any block under pressure	A pinned block cannot be evicted while any lease is live
Scope	Lives inside one engine	A cross-engine arbiter over multiple engines sharing one memory pool
Admission	No awareness of other work	New work is budgeted around the pinned master

The two guarantees, enforced by contract and proven by tests

Pinned-never-evicted. A pinned master is never evicted, for any reason, under any pressure. This is correct by construction, because no eviction path exists for a leased block, and it is verified at the transport layer.

One-prefill-per-model. The master is prefilled at most once per distinct model, ever. This is proven directly: ten concurrent leases for a single model produce exactly one measured prefill, not ten.

05 The shared-memory claim, stated precisely

This is the most important section in the document. The claim is specific, and its precision is its strength. Two different things are shared, and they must never be confused.

The data store — one copy, read by all

The telemetry lives in a single store. Every agent reads that one copy in its own way, extracting only the patterns its role requires. No agent duplicates the raw data. This part is straightforward and total.

The model's memory — shared per distinct model

Each model turns context into a key-value cache, a set of tensors whose exact shape is fixed by that model's own weights and dimensions. Here lies the subtle heart of the design. Agents running on the **same** model share one master: the Correlator and the Reporter both run on gpt-oss-120b, so they read a single pinned copy, and the duplication that would otherwise exist between them simply vanishes. Agents on **different** models cannot share, because a cache built by one model is mathematically meaningless to another. Each distinct model therefore holds its own pinned master.

> **The sharing unit is the distinct model, never the family.** Even gpt-oss-20b and gpt-oss-120b, despite the shared name, are different models with incompatible cache geometry. They do not share. This precision is deliberate, and it is what keeps the claim honest.

06 The mathematical constraint — why four models need 192 GB

Why can two different models never share one cache? Because a cache is not text. It is a body of numeric tensors produced by a specific model's weights. Different models have different dimensions, so the tensors are physically different shapes. And even where shapes happen to align, the numbers themselves are drawn from each model's own weights, so one model's cache is not another language to a second model. It is noise. This is not a limitation of present-day science that a future breakthrough will lift. It is arithmetic, as fixed as the fact that one car's key will not turn in another car's lock.

That constraint is precisely why the project needs 192 GB. Because different models cannot share memory, each distinct model must hold its own master resident at the same time.

Model	Agent(s)	Resident weights
gpt-oss-20b	Triage	~14 GB
gpt-oss-120b	Correlator and Reporter (shared)	~76 GB
Llama 3.3 70B	Hunter	~48 GB
Qwen3-32B	Topology	~24 GB
Total resident weights, before a single byte of context		**~162 GB**

> **The capacity thesis, in one line.** One hundred and sixty-two gigabytes of weights does not fit an 80 GB card at all. It fits an AMD Instinct MI300X, with headroom to spare. The argument is absolute, not relative: it is about fitting or not fitting, not about faster or slower.

07 The domain, and who this is for

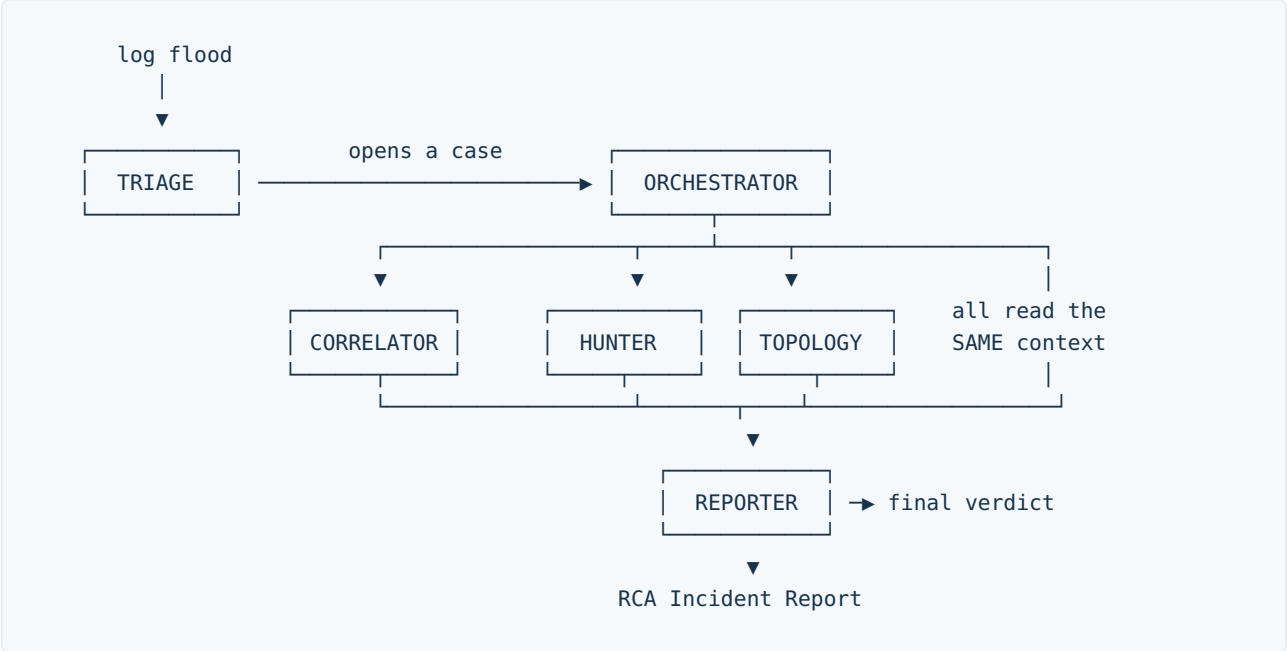
The domain is AIOps: incident root-cause analysis. A production system fails, thousands of log lines flood in, and buried among them are the few lines that explain why. The product serves the site reliability engineer investigating a live incident, the platform and DevOps engineer, and above all the on-call engineer paged at three in the morning who must find the root cause in a flood of logs within minutes.

A human on-call engineer averages hours to find a root cause. Remanon's five agents, sharing one pinned memory per model, do it in seconds. The domain fits the architecture for two precise reasons: the context must be shared, because the whole team investigates one incident, and it must be long and stable, because the evidence body is large and reused. Those are exactly the two conditions any domain must satisfy to plug into the core.

08 The agents, the flow, and the output

Agent	What it does with the data	Model
Triage	Classifies severity and opens the case	gpt-oss-20b
Correlator	Builds the causal hypothesis, what led to what	gpt-oss-120b
Hunter	Writes queries to pull hard evidence from the store	Llama 3.3 70B
Topology	Maps the blast radius, which nodes and links were hit	Qwen3-32B
Reporter	Synthesizes the final verdict	gpt-oss-120b

The Correlator and the Reporter run on the same model, so they share a single pinned master. That is the amortization proven inside one run: two agents, one resident copy. Deciding who runs and when is the Orchestrator, a deterministic state machine that routes the case, runs the investigators concurrently, waits for them, and hands their results to the Reporter. The agents never message one another at random. The Orchestrator conducts.

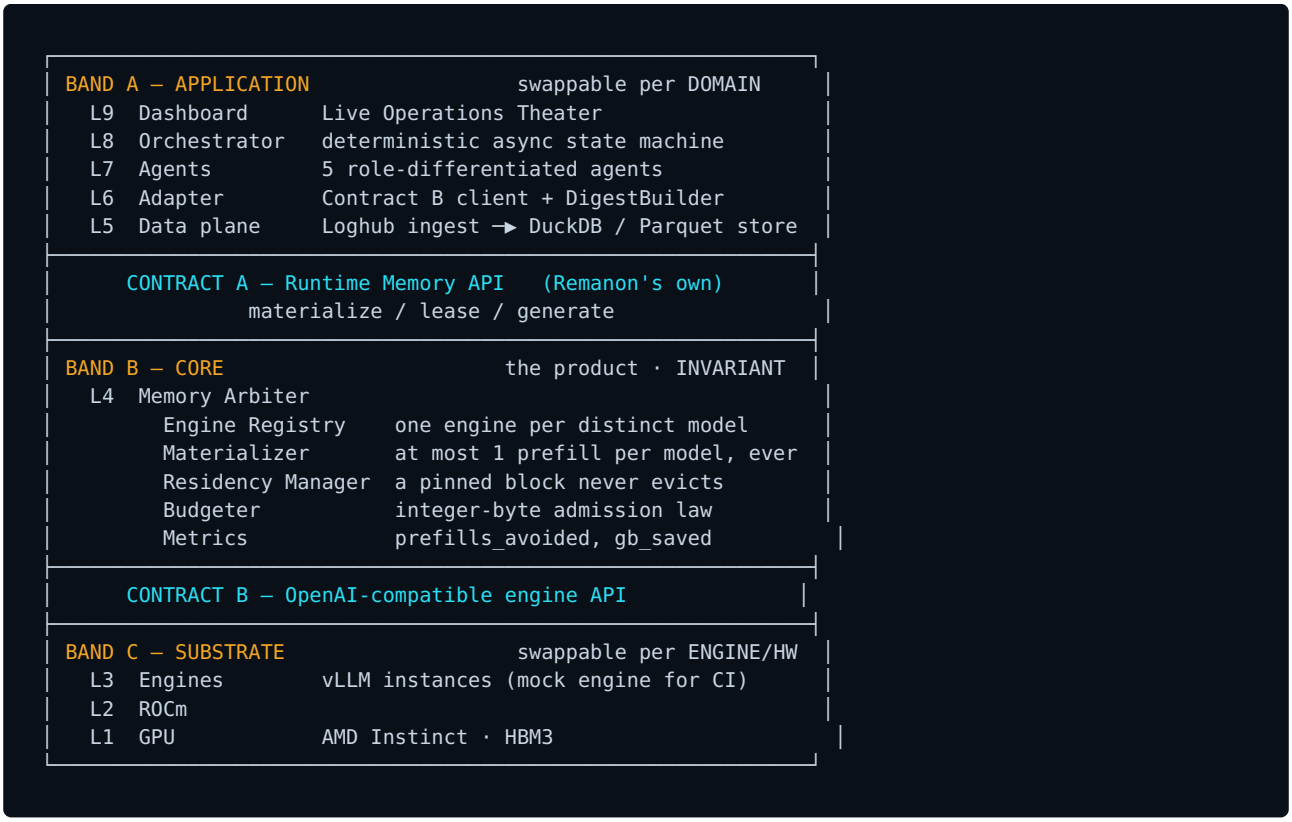


The output — a real incident report the engineer can take away

The final verdict is a root-cause analysis report carrying the severity, the problem, the root cause, the supporting evidence, the blast radius, and a recommendation. It is exported live as CSV and Markdown, ready for formal delivery, and routed to the team that needs it: the on-call engineer, or the networking team when the cause is a network fault. On the measured system, the engineer downloads the full case ledger directly from the live dashboard, every field drawn from the recorded run.

Real demo output. From a genuine flood of production log records, the system opened and classified a full set of live cases on AMD hardware, each with its real severity, its affected nodes, and its own model-written reasoning, then made the complete ledger available as a downloadable incident report. The run was live, the engine real, the verdicts genuine. MEASURED

09 Architecture — three bands, nine layers, two contracts



The two contracts

Contract A, the Runtime Memory API of materialize, lease, and generate, is Remanon's own invention. It carries both guarantees, and the agents speak only to it, never touching an engine directly. Contract B is the OpenAI-compatible engine API that vLLM already serves and that the mock engine implements for GPU-free continuous integration. The core is the single component that translates one contract into the other.

The core — four components, four invariants

Component	Responsibility	Invariant enforced
Engine Registry	One engine per distinct model	No agent addresses an engine directly
Materializer	One-time prefill to a model-specific cache	At most one prefill per model, ever
Residency Manager	Issues leases, enforces pinning	A leased block is never evicted
Budgeter	Partitions the memory pool	Pressure hits deltas, never masters

Beneath them sits the store, a single copy of the logs in DuckDB and Parquet, queried by every agent and built from real Loghub production data spanning HDFS, BGL, Thunderbird, Spark, and Hadoop.

10 Why it is a Lego — the domain-agnostic core

Remanon is built as three interlocking bands. Band A swaps per domain, carrying AIOps today and any agent team tomorrow. Band C swaps per engine and per hardware substrate. Band B, the core, never changes. The memory arbiter neither knows nor cares what the agents investigate. Today it powers incident analysis; the identical core plugs into any multi-agent, multi-model workload without a line of its logic changing. Proven on one team today, it is open to any team tomorrow.

11 Measured evidence on AMD — the real numbers

Captured on a live AMD ROCm instance, gfx1100 with 48 GB, serving gpt-oss-20b under vLLM. Raw logs are archived in the repository. Nothing here is fabricated or simulated.

Evidence	Value	Provenance
GPU and architecture	AMD gfx1100, 48 GB VRAM	MEASURED
Model weights resident	14.3 GiB	MEASURED
Key-value cache available	26.67 GiB	MEASURED
Cache capacity in tokens	582,576 tokens	MEASURED
VRAM used while serving	46.02 GB of 48 GB	MEASURED
Shared-context reuse	cold 10.635 s to warm 1.839 s — 5.8× faster	MEASURED
Engine prefix-cache hit rate	66.3%	MEASURED
Prefills avoided across the run	110	MEASURED
Evictions of a pinned master	0	MEASURED

> **Two independent signals confirm the same fact.** Our own timed measurement reports a 5.8-times speedup, and the engine's internal counter reports a 66.3% prefix-cache hit rate. Shared-context reuse is real on AMD, and it saves. The mechanism is measured, not asserted, and the logic beneath it is proven by **170 automated tests**, including transport-layer contract tests that prove both guarantees directly. This is real code, not slides.

12 The two versions — measured reality and research vision

The allocated hardware was a 48 GB gfx1100, not the 192 GB MI300X the design targets. Rather than obscure that gap, the project presents two honestly labelled views of the same system at two scales.

Measured Reality — what actually ran MEASURED

One model, gpt-oss-20b, served live on AMD gfx1100, processing a genuine flood of production logs into real, classified verdicts, with every headline number taken straight off the hardware: 46 of 48 GB in use, 14.3 GiB of weights, 26.67 GiB of cache, a 5.8-times reuse speedup, a 66.3% hit rate, and a downloadable incident ledger. On 48 GB, the full four-model team does not fit, and that very impossibility is the proof of the capacity thesis.

Research Vision — the full system on the right hardware COMPUTED

The complete architecture of five agents across four models, one pinned master per model, resident together on 192 GB of HBM3. This view is grounded in the measured mechanism, the 170 tests proving the logic, and a transparent arithmetic projection of the memory budget, and it is labelled throughout as projected, never as measured.

Why only one model ran live. The four models together require roughly 162 GB and physically cannot co-reside on a 48 GB card. Running them together on the provided hardware is impossible, not a choice we declined to make. So the mechanism is measured on what fits and mathematically projected, from measured numbers and passing tests, to the hardware it was designed for. This is the standard scientific treatment of a resource constraint: measured where possible, transparently extrapolated where not, and nothing claimed as measured that was not.

13 Why AMD is the hero

Remanon's competitive argument is capacity, not speed. It rests on the absolute difference between a multi-agent team fitting, or not fitting, in one memory pool. With a small memory, the idea fails outright: four models overflow an 80 GB card and the team cannot even load. With a large memory, the idea works: the 192 GB HBM3 of the AMD Instinct MI300X is the enabling substrate that lets the impossible fit.

This is AMD's own strategic story, told back to AMD. Memory-rich, open-model, inference-bound workloads are exactly where the MI300X wins, and Remanon is a workload that genuinely needs 192 GB. There is no more convincing demonstration of meaningful use of AMD than a system whose entire feasibility is decided by the memory only AMD brings.

14 The economic argument — cost that follows from capacity

Because the capacity argument is absolute, it converts directly into cost. A four-model team that does not fit an 80 GB card must be split across several smaller GPUs, each rented by the hour, each running in parallel. The same team fits a single MI300X. The saving is not a marginal tuning gain. It is a structural consequence of removing the duplication that forced the extra hardware in the first place.

	Without Remanon	With Remanon
GPUs required	Several smaller cards, because 162 GB does not fit 80 GB	One MI300X, 192 GB
Relative hourly cost	Multiplied by the number of cards	A single card's rate
Direction of saving	—	A substantial reduction, compounding over continuous operation

Anchored in what was measured. On real 48 GB hardware, the Triage path alone avoided 110 prefills and saved 10 GB of duplicated cache against a per-agent baseline. Those are measured savings on a single agent; the architecture multiplies them with every additional agent that shares a model. MEASURED

15 Market proof

The problem Remanon attacks, shared context in multi-agent inference, is a live and funded market rather than a speculative one. Memory-runtime startups working adjacent claims have collectively raised substantial venture funding, which establishes both that the pain is real and that buyers are willing to pay to relieve it. Remanon's differentiator over all of them is singular and defensible: a guaranteed, cross-engine residency contract that opportunistic prefix caches cannot provide. Where others optimize by chance, Remanon guarantees by construction.

16 Deployment and the live site

The judged surface is deployed as a static website on GitHub Pages, requiring no server and no cloud credits, permanent and free at **nevineakf.github.io/remanon**. A GitHub Actions workflow regenerates the site from the honest showcase build on every push, and it refuses to publish if the measured or computed provenance labels are ever missing. That integrity gate is automated, so the honesty of the site is enforced by the build itself, not merely promised.

The site presents the two-version story directly. A visitor chooses between the Measured Reality path, where every figure came off the hardware, and the Research Vision path, where the full system is projected on the target 192 GB card, complete with an interactive memory allocator that computes, in real arithmetic on real model sizes, exactly when a team fits an 80 GB card, when it fits 192 GB, and when it overflows. The dashboard is a recorded replay of a genuine run, honest by construction, and it renders instantly for any viewer without a server behind it.

17 Defense — the hardest questions, answered

Any technical panel will ask at least one of these. Knowing the weak points before they are raised, and having a clear answer ready, is what earns trust.

Q **This is just prefix caching. vLLM already does it.**

vLLM's prefix caching is best-effort, evictable under pressure, and confined to a single engine. Remanon guarantees non-eviction across multiple engines sharing one memory pool. The difference is guarantee versus hope, and no component in today's stack plays the cross-engine guaranteed-residency role that Remanon fills.

Q **Why can't you share memory between different models?**

Because it is mathematically impossible. A cache's shape is fixed by the model's weights, and one model's cache is noise to another. That is precisely why each distinct model gets its own master, and precisely why the full team requires 192 GB.

Q **You ran one model. How do you claim four?**

The mechanism was measured on the hardware that was available, the gfx1100, and mathematically projected, with tests and a stated methodology, to the hardware it targets, the MI300X. This is the standard scientific treatment of a resource constraint. Nothing is claimed as measured that was not measured.

Q **The sharing is only between two agents. Isn't that weak proof?**

The mechanism scales with team size. Five agents is a proof of concept, and the value grows with the number of agents per model: a larger deployment with many agents on a single model saves proportionally more. The saving is a function of agents-per-model, and the underlying logic is proven independent of that count by the tests.

Q **Why AMD, and not NVIDIA?**

Because the argument is absolute capacity, not speed. The team fits or it does not, a binary decided entirely by memory, and AMD leads on large-memory inference with the 192 GB of the MI300X.

Q **Is the data real, or simulated?**

The data is real Loghub production logs from HDFS, BGL, Thunderbird, Spark, and Hadoop, replayed for reproducibility and ground truth rather than streamed live. The data plane is swappable to a live source such as Fluentd, Vector, or Kafka without touching the core. We simulate the source, never the data.

Q **What is the real market?**

Any multi-agent, multi-model team. The core is domain-agnostic by design, carrying AIOps today and any domain tomorrow, and adjacent startups are already funded on narrower versions of the same claim.

18 Honest constraints, stated openly

Nothing here is hidden, because transparency is the strength of the project rather than a concession it makes. The hardware allocated was a 48 GB gfx1100 on a limited quota, not the 192 GB MI300X the design targets. The single-model numbers are measured, while the four-model, 192 GB budget is computed from those measured numbers by a stated methodology. The data is real Loghub production logs, replayed for reproducibility rather than fed live, and swappable to a live source without touching the core. And the memory sharing occurs per distinct model, between same-model agents, never across different models, which is a mathematical fact honestly bounded rather than a limitation quietly hidden.

> **The foundation, in one sentence.** Every figure states its provenance. Measured where possible, transparently projected where not, nothing fabricated and everything labelled. That combination is the honest, defensible foundation on which the entire project stands.